

Atividade Prática: Construindo e Testando uma Calculadora em Python

Introdução

Esta atividade tem como objetivo consolidar seus conhecimentos em programação Python orientada a objetos e na escrita de testes unitários utilizando o framework Pytest. Você irá desenvolver uma classe de calculadora básica e, em seguida, criar testes robustos para garantir a correção de suas operações.

Objetivos de Aprendizagem

Ao final desta atividade, você será capaz de:

Implementar uma classe Python com métodos para operações matemáticas básicas. Aplicar os princípios de tratamento de erros em Python.

Escrever testes unitários eficazes para métodos de classe usando Pytest. Identificar e testar diferentes cenários (casos de sucesso, casos de falha, casos de borda).

Recursos de Apoio

Pytest Cheat Sheet: Utilize o `pytest_cheat_sheet.md` (ou `pytest_cheat_sheet.pdf`) como referência rápida para os comandos e sintaxe do Pytest.

Documentação Python: Para dúvidas sobre a sintaxe da linguagem e classes.

Parte 1: Implementação da Classe Calculadora

Tarefa

Crie um arquivo Python chamado `calculadora.py` e implemente uma classe `Calculadora` que contenha os seguintes métodos:

1. `soma(a, b)` : Retorna a soma de `a` e `b` .
2. `subtracao(a, b)` : Retorna a subtração de `b` de `a` .

3. `multiplicacao(a, b)` : Retorna o produto de a e b .

4. `divisao(a, b)` : Retorna a divisão de a por b .

Tratamento de Erro: Este método deve levantar uma exceção `ValueError` com a mensagem “Não é possível dividir por zero!” se b for zero.

Exemplo de Estrutura da Classe (`calculadora.py`)

```
class Calculadora:
    def soma(self, a, b):
        # Implemente aqui
        pass

    def subtracao(self, a, b):
        # Implemente aqui
        pass

    def multiplicacao(self, a, b):
        # Implemente aqui
        pass

    def divisao(self, a, b):
        # Implemente aqui, incluindo o tratamento de erro
        pass
```

Parte 2: Testes Unitários com Pytest

Tarefa

Crie um arquivo Python chamado `test_calculadora.py` na mesma pasta de `calculadora.py` e escreva testes unitários para todos os métodos da classe `Calculadora` utilizando o `Pytest`. Certifique-se de cobrir os seguintes cenários:

1. `soma` :

Testar com números positivos.

Testar com números negativos.

Testar com zero.

2. subtracao :

Testar com números positivos.
Testar com números negativos.
Testar com zero.

3. multiplicacao :

Testar com números positivos.
Testar com números negativos.
Testar com zero.

4. divisao :

Testar com números positivos.
Testar com números negativos.
Testar com zero (deve verificar se a exceção ValueError é levantada).
Testar com resultados decimais.

Exemplo de Estrutura de Teste (test_calculadora.py)

```
import pytest
from calculadora import Calculadora

# Instância da calculadora para ser usada nos testes
@pytest.fixture
def calc():
    return Calculadora()

def test_soma_positivos(calc):
    assert calc.soma(5, 3) == 8

def test_divisao_por_zero(calc):
    with pytest.raises(ValueError, match="Não é possível dividir por zero"):
        calc.divisao(10, 0)

# Adicione mais testes para todos os métodos e cenários!
```

Execução dos Testes

Após implementar a classe e os testes, navegue até a pasta do projeto no terminal e execute:
`pytest -v`

Verifique se todos os seus testes passaram. Se algum falhar, analise a saída do Pytest para identificar o problema e corrija-o na sua classe Calculadora ou nos seus testes.

Entrega

Entregue os arquivos `calculadora.py` e `test_calculadora.py` com suas implementações e testes. Certifique-se de que todos os testes passem ao serem executados com `pytest -v`.